



CODE

OUTPUT

```
1  #include <stdio.h>
2  #define NUM_CELLS 4 // change this to 8, 12, 16 etc.
3  int main()
4  {
5      float cellVoltage[NUM_CELLS];
6      float highest, lowest;
7      float imbalance, previousImbalance;
8      float soc;
9      float threshold;
10     int highestCell = 0, lowestCell = 0;
11
12     printf("==== Modular Battery Management System====\n");
13
14     // Read cell voltages
15     printf("\n Enter voltage of %d cells:\n", NUM_CELLS);
16
17
18     for(int i=0; i<NUM_CELLS; i++)
19     {
20         printf("cell %d:", i + 1);
21         scanf("%f", &cellVoltage[i]);
22     }
23
24     //read state of charge
25     printf("\nEnter Battery State of Charge (0-100%%): ");
26     scanf("%f", &soc);
27
28     // Read previous imbalance
29     printf("Enter Previous Voltage Imbalance (V): ");
30     scanf("%f", &previousImbalance);
31
32     // Initialize highest and lowest
33     highest = cellVoltage[0];
34     lowest = cellVoltage[0];
35
36     // Find highest and lowest cell
37     for(int i = 1; i < NUM_CELLS; i++)
38     {
39         if(cellVoltage[i] > highest)
40         {
41             highest = cellVoltage[i];
42             highestCell = i;
43         }
44         if(cellVoltage[i] < lowest)
45         {
46             lowest = cellVoltage[i];
47             lowestCell = i;
48         }
49     }
50     // Calculate imbalance
51     imbalance = highest - lowest;
52 }
```



CODE

OUTPUT

```
33 highest = cellVoltage[0];
34 lowest = cellVoltage[0];
35
36 // Find highest and lowest cell
37 for(int i = 1; i < NUM_CELLS; i++)
38 {
39     if(cellVoltage[i] > highest)
40     {
41         highest = cellVoltage[i];
42         highestCell = i;
43     }
44     if(cellVoltage[i] < lowest)
45     {
46         lowest = cellVoltage[i];
47         lowestCell = i;
48     }
49 }
50 // Calculate imbalance
51 Imbalance = highest - lowest;
52
53 // Adaptive threshold
54 if(soc >= 80)
55     threshold = 0.03;
56 else if(soc >= 50)
57     threshold = 0.05;
58 else
59     threshold = 0.08;
60 // Display results
61 printf("\n===== RESULTS =====\n");
62
63 printf("Strongest Cell : Cell %d (%.2f V)\n",
64        highestCell + 1, highest);
65
66 printf("Weakest Cell : Cell %d (%.2f V)\n",
67        lowestCell + 1, lowest);
68
69 printf("Voltage Imbalance : %.2f V\n", Imbalance);
70 if(Imbalance > previousImbalance)
71     printf("Trend : Imbalance Increasing\n");
72 else if(Imbalance < previousImbalance)
73     printf("Trend : Imbalance Decreasing\n");
74 else
75     printf("Trend : No Change\n");
76
77 printf("Adaptive Threshold : %.2f V\n", threshold);
78 if(Imbalance > threshold)
79     printf("Status : WARNING! Battery Balancing Required.\n");
80 else
81     printf("Status : Battery is Healthy.\n");
82
83 return 0;
84 }
```



TAB



==== Modular Battery Management System =====

Enter voltage of 4 cells:

Cell 1: 3.72

Cell 2: 3.68

Cell 3: 3.75

Cell 4: 3.70

Enter Battery State of Charge (0-100%): 85

Enter Previous Voltage Imbalance (V): 0.03

===== RESULTS =====

Strongest Cell : Cell 3 (3.75 V)

Weakest Cell : Cell 2 (3.68 V)

Voltage Imbalance : 0.07 V

Trend : Imbalance Increasing

Adaptive Threshold : 0.03 V

Status : WARNING! Battery Balancing Required.

[Program finished]



GIF



1

2

3

4

5

6

7

8

9

0

q

w

e

r

t

y

u

i

o

p

a

s

d

f

g

h

j

k

l



z

x

c

v

b

n

m



?123

,

.





new*



```
1 #include <stdio.h>
2
3 #define NUM_CELLS 4 // Change to 8, 12, or
  16 if required
4
5 void findCells(float cellVoltage[], float *highest,
  float *lowest,
6               int *highestCell, int *lowestCell)
7 {
8     *highest = cellVoltage[0];
9     *lowest = cellVoltage[0];
10    *highestCell = 0;
11    *lowestCell = 0;
12
13    for(int i=1;i<NUM_CELLS;i++)
14    {
15        if(cellVoltage[i]>*highest)
16        {
17            *highest=cellVoltage[i];
18            *highestCell=i;
19        }
20
21        if(cellVoltage[i]<*lowest)
22        {
23            *lowest=cellVoltage[i];
24            *lowestCell=i;
25        }
26    }
27 }
28
29 float calculateImbalance(float highest,float lowest)
30 {
31     return highest-lowest;
32 }
33
34 float getThreshold(float soc)
35 {
36     if(soc>=80)
37         return 0.03;
38     else if(soc>=50)
39         return 0.05;
40     else
```

Tab

{

:

;





new*



```
37     return 0.05;
38     else if(soc>=50)
39         return 0.05;
40     else
41         return 0.08;
42 }
43
44 void displayResult(int highestCell,int lowestCell,float
highest,float lowest,
45                 float imbalance,float previous,
float threshold)
46 {
47     printf("\n===== RESULTS
=====\\n");
48     printf("Strongest Cell : Cell %d (%.2f V)\\n",
highestCell+1,highest);
49     printf("Weakest Cell   : Cell %d (%.2f V)\\n",
lowestCell+1,lowest);
50     printf("Voltage Imbalance : %.2f V\\n",imbalance);
51
52     if(imbalance>previous)
53         printf("Trend : Imbalance Increasing\\n");
54     else if(imbalance<previous)
55         printf("Trend : Imbalance Decreasing\\n");
56     else
57         printf("Trend : No Change\\n");
58
59     printf("Adaptive Threshold : %.2f V\\n",
threshold);
60
61     if(imbalance>threshold)
62         printf("Status : WARNING! Battery
Balancing Required.\\n");
63     else
64         printf("Status : Battery is Healthy.\\n");
65 }
66
67 int main()
68 {
69     float cellVoltage[NUM_CELLS];
70     float highest,lowest;
71     float imbalance;
```

Tab

{

:

;





new*



```
71     float imbalance;
72     float previousImbalance;
73     float soc;
74     float threshold;
75     int highestCell,lowestCell;
76
77     printf("==== Modular Battery Management
System ====\\n");
78
79     printf("Enter voltage of %d cells:\\n",
NUM_CELLS);
80
81     for(int i=0;i<NUM_CELLS;i++)
82     {
83         printf("Cell %d: ",i+1);
84         scanf("%f",&cellVoltage[i]);
85     }
86
87     printf("Enter Battery State of Charge (0-
100%%): ");
88     scanf("%f",&soc);
89
90     printf("Enter Previous Voltage Imbalance (V):
91
92     scanf("%f",&previousImbalance);
93
94     findCells(cellVoltage,&highest,&lowest,
&highestCell,&lowestCell);
95
96     imbalance=calculateImbalance(highest,lowest);
97
98     threshold=getThreshold(soc);
99
100    displayResult(highestCell,lowestCell,highest,
lowest,
101                  imbalance,previousImbalance,
threshold);
102    return 0;
103 }
```

Tab

{

}

:

;





TAB



==== Modular Battery Management System =====

Enter voltage of 4 cells:

Cell 1: 3.72

Cell 2: 3.68

Cell 3: 3.75

Cell 4: 3.70

Enter Battery State of Charge (0-100%): 85

Enter Previous Voltage Imbalance (V): 0.03

===== RESULTS =====

Strongest Cell : Cell 3 (3.75 V)

Weakest Cell : Cell 2 (3.68 V)

Voltage Imbalance : 0.07 V

Trend : Imbalance Increasing

Adaptive Threshold : 0.03 V

Status : WARNING! Battery Balancing Required.

[Program finished]

